Практическая работа № 2.2 Контроллер и представление на PHP.

Цель: Оптимизация разработки при помощи архитектурных паттернов разработки.

Теория:

Контроллер — связующее звено, соединяющее модели, виды и другие компоненты в рабочее приложение. Контроллер отвечает за обработку запросов пользователя. Контроллер не должен содержать SQL-запросов. Их лучше держать в моделях. Контроллер не должен содержать HTML и другой разметки. Её стоит выносить в виды.

В хорошо спроектированном MVC-приложении контроллеры обычно очень тонкие и содержат только несколько десятков строк кода. Чего, не скажешь о Stupid Fat Controllers (SFC) в CMS Joomla. Логика контроллера довольно типична и большая ее часть выносится в базовые классы.

Модели, наоборот, очень толстые и содержат большую часть кода, связанную с обработкой данных, т.к. структура данных и бизнес-логика, содержащаяся в них, обычно довольно специфична для конкретного приложения

Практика:

Задание 1.

app/Core/Controller.php

```php
<?php

namespace App\Core;

abstract class Controller
{
    // Пока пустой — наполним на этапе View
}
```

app/Controllers/ArticleController.php

```php
<?php

namespace App\Controllers;

use App\Core\Controller;
```

```php
class ArticleController extends Controller
{
    public function index(): void
    {
        // Захардкоженные данные — БД ещё нет!
        $articles = [
            ['id' => 1, 'title' => 'Первая статья'],
            ['id' => 2, 'title' => 'Вторая статья'],
            ['id' => 3, 'title' => 'Третья статья'],
        ];

        echo '<h1>Все статьи</h1>';
        foreach ($articles as $a) {
            echo "<p>{$a['title']} — <a href='/article/{$a['id']}'>читать</a>
</p>";
        }
    }

    public function show(int $id): void
    {
        // Пока просто эхо
        echo "<h1>Статья #{$id}</h1>";
        echo '<p>Здесь будет содержимое статьи.</p>';
        echo '<p><a href="/">← Назад</a></p>';
    }
}
```

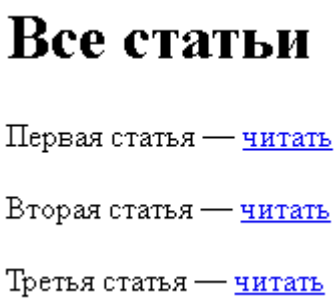


Рисунок 1 - список статей из контроллера

index.php

```php
$router->get('/', [ArticleController::class, 'index']);
$router->get('/article/{id}', [ArticleController::class, 'show']);
```

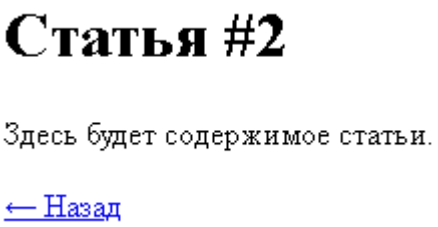


Рисунок 2 - страница одной статьи

Теоретические вопросы:

- Что делает контроллер?

Ответ: Контроллер принимает запрос и решает, что вывести на страницу.

- Как вы думаете, что мы с вами сделали не правильно ?

Ответ: HTML напечатан через echo прямо в контроллере. Разметка и логика страницы оказались в одном классе.

- Что не так с этим echo?

Ответ: Чтобы поменять внешний вид, приходится править PHP-класс. Общую шапку так пришлось бы копировать в каждый метод.

Теория:

Вид — используется для задания внешнего отображения данных, полученных из контроллера и модели.

Виды содержат HTML-разметку и небольшие вставки PHP-кода для обхода, форматирования и отображения данных.

Не должны напрямую обращаться к базе данных. Этим должны заниматься модели.

Не должны работать с данными, полученными из запроса пользователя. Эту задачу должен выполнять контроллер.

Может напрямую обращаться к свойствам и методам контроллера или моделей, для получения готовых к выводу данных.

Виды обычно разделяют на общий шаблон, содержащий разметку, общую для всех страниц (например, шапку и подвал) и части шаблона, которые используют для отображения данных выводимых из модели или отображения форм ввода данных.

Задание 2.

Идея: контроллер готовит данные, view рисует.

app/Core/View.php

```php
<?php

namespace App\Core;

class View
{
    public static function render(string $template, array $data = []): void
```

```
    {
        extract($data); // $articles / $article станут переменными

        $viewFile = __DIR__ . '/../Views/' . $template . '.php';
        if (!file_exists($viewFile)) {
            http_response_code(500);
            exit("View not found: {$template}");
        }

        ob_start();
        require $viewFile;
        $content = ob_get_clean();  // содержимое шаблона

        require __DIR__ . '/../Views/layout.php'; // обёртка
    }
}
```

app/Core/Controller.php (наполняем)

```
<?php

namespace App\Core;

abstract class Controller
{
    protected function view(string $template, array $data = []): void
    {
        View::render($template, $data);
    }
}
```

app/Views/layout.php

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <title>MVC Demo Blog</title>
    <style>
        body { font-family: sans-serif; max-width: 800px; margin: 40px auto;
padding: 0 20px; }
        .article { border-bottom: 1px solid #eee; padding: 15px 0; }
        a { color: #0066cc; text-decoration: none; }
        a:hover { text-decoration: underline; }
    </style>
</head>
<body>
    <header>
        <h1><a href="/"> MVC Blog</a></h1>
        <hr>
    </header>
    <main><?= $content ?></main>
</body>
</html>
```

app/Views/articles/index.php

```
<h2>Все статьи</h2>

<?php foreach ($articles as $article): ?>
```

```
    <div class="article">
        <h3><?= htmlspecialchars($article['title']) ?></h3>
        <a href="/article/<?= $article['id'] ?>">Читать →</a>
    </div>
<?php endforeach; ?>
```

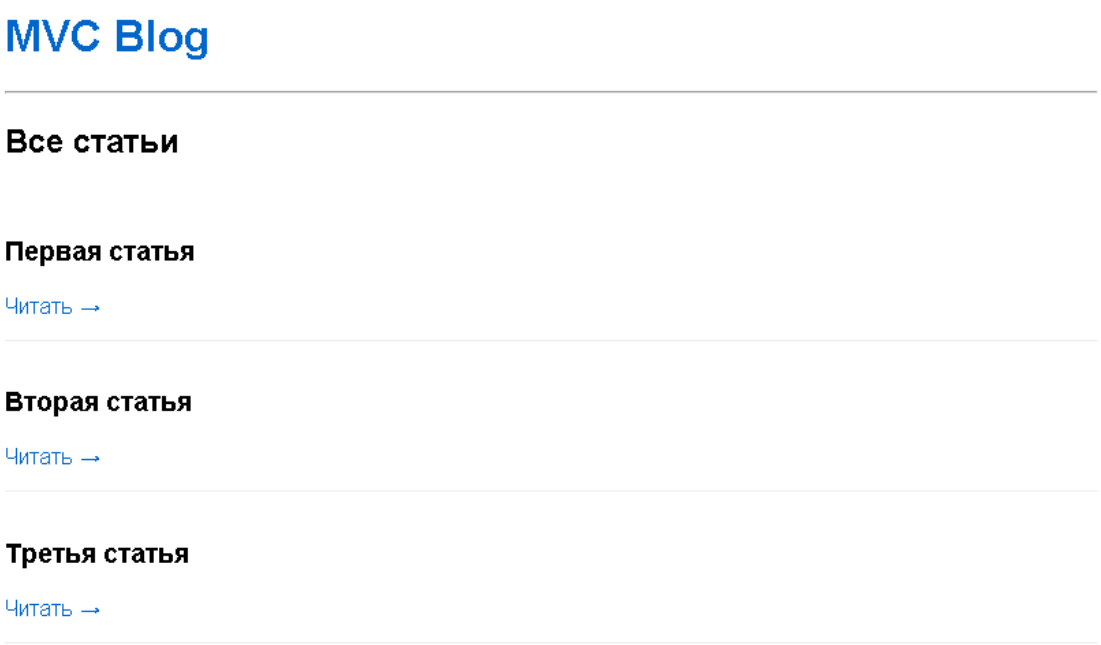


Рисунок 3 - список статей в общем шаблоне

app/Views/articles/show.php

```
<a href="/">← Назад</a>
<h2><?= htmlspecialchars($article['title']) ?></h2>
<p><?= nl2br(htmlspecialchars($article['content'])) ?></p>
```

app/Core/Controller.php

```
<?php

namespace App\Controllers;

use App\Core\Controller;

class ArticleController extends Controller
{
    public function index(): void
    {
        $articles = [
            ['id' => 1, 'title' => 'Первая статья', 'content' => '...'],
            ['id' => 2, 'title' => 'Вторая статья', 'content' => '...'],
            ['id' => 3, 'title' => 'Третья статья', 'content' => '...'],
        ];

        $this->view('articles/index', ['articles' => $articles]);
    }

    public function show(int $id): void
    {
        $article = [
```

```
            'id' => $id,
            'title' => "Статья #{$id}",
            'content' => 'Здесь будет содержимое статьи. Пока это заглушка.',
        ];

        $this->view('articles/show', ['article' => $article]);
    }
}
```

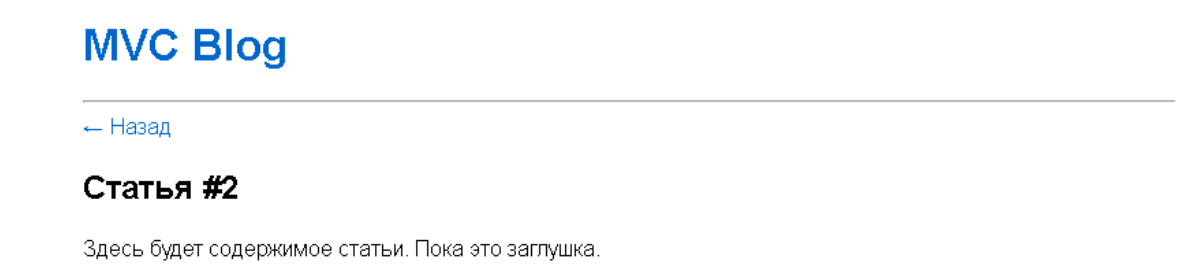


Рисунок 4 - страница статьи в общем шаблоне

Теоретические вопросы:

1. Что именно делает контроллер после рефакторинга?

Ответ: Собирает массив и вызывает $this->view() с именем шаблона.

2. Чем теперь занят контроллер, если из него убрали HTML?

Ответ: готовит данные и выбирает шаблон. Разметку он больше не пишет.

3. Что осталось в контроллере, а что ушло в шаблон?

Ответ: В контроллере остались данные и имя шаблона. В шаблон ушли заголовки, ссылки и абзацы.

4. Почему layout.php один, а шаблонов может быть много?

Ответ: Шапка одна на все страницы, середина у каждой своей. Новый экран — это новый файл в Views, каркас тот же.

5. Как layout.php связан с ob_start() и ob_get_clean()?

Ответ: ob_start () задерживает вывод шаблона, ob_get_clean() кладёт его в переменную $content. layout.php вставляет эту переменную в страницу.

6. Что было бы, если бы каждый шаблон содержал свой <html>, <head> и <body>?

Ответ: Шапку и стили пришлось бы копировать в каждый шаблон.

7. Что будет, если завтра дизайнер захочет поменять меню?

Ответ: Меню меняется в одном файле, layout.php. Остальные шаблоны трогать не нужно.

8. А откуда взять статьи, если их станет 10 000?

Ответ: из базы. Держать тысячи статей массивом в контроллере нельзя, чтение данных переносится в модель.

ЛИТЕРАТУРА

1. https://blog.skillfactory.ru/glossary/mvc/ - Что такое MVC.
2. https://www.php.net/manual/ru/ - Официальная русская документация по PHP.
3. https://metanit.com/php/tutorial- Учебник по PHP.
4. https://habr.com/ru/articles/150267/ - Еще один способ реализации MVC.